Agile! The Good, the Hype, and the Ugly (Bertrand Meyer)

Book Summary

Patrick Bucher

2026-02-16

Contents

This is a practical book that enables the reader to benefit from the good ideas of agile methods while staying away from the bad ones. Agile methods are a mix of horrible and great ideas. In order to spare the reader from having to try out Scrum, Extreme Programming, Lean Software, and Crystal all on his own, this book provides a description and assessment of their underlying ideas.

First, those methods are described without the usual sermons and anecdotes. Second, the underlying ideas are scrutinized and assessed, thereby separating the useful from the harmful ones. Agile methods and their texts put three difficoulties in the reader's way:

1. *Partisanship*: While it's common for their inventors to argue in favour of their inventions, proponents of agile methods ignore the rules of rational discourse and ask the reader to join their cult, which is inappropriate for engineering problems.
2. *Intimidation*: Agile texts dismiss previous approaches to software development as outdated and their users as rigid and backwards. Who wants to argue against "agile" when all of this word's opposites have negative meanings?
3. *Extremism*: Proponents of some agile methods insist on the application of a method in its entirety, which discourages a more differentiated view on the techniques that make up a specific method.

While previous books criticizing agile methods were rather timid, this book will call out the bad ideas without undue dereference.

The first chapter is a summary of agile ideas. In the second chapter, the rhetoric of agile texts is analyzed. Chapter three is a description of traditional plan-based software development—to which agile methods are the antidote. Chapters four to eight review agile ideas: principles, roles, practices, and artifacts.

Those chapters do not focus on individual methods, but on their many commonalities. Scrum, Lean, XP, and Crystal are then presented in chapter nine as combinations of those principles already discussed, each with their one single big idea emphasized.

Chapter ten describes precautions organizations should take when adopting agile methods. Chapter eleven is the final assessment; it classifies agile ideas into three categories: the good, the hype, and the ugly.

This book is not a comprehensive guide to software development, but a critique of existing approaches. Authors usually argument by gut feeling, by experience (e.g. projects saved by agile approaches and ruined by ignoring them), logical reasoning, and, ideally, empirical analysis (for which there is usually too little data available). This book relies mostly on analytical reasoning, combined with personal experience and anecdotes for illustrative purposes.

1 Overview

While agile ideas date back to the 1990s, the movement gained traction with the publication of the *Manifesto for Agile Software Development* in 2001, which was signed by many proponents of existing agile ideas. Agile software development is not a single method, but a collection of ideas grouped together to methodologies such as *Extreme Programming*, *Lean Software Development*, *Crystal*, and *Scrum*, which all select and prioritize their own subset of agile ideas. However, there are some common core characteristics to those methods:

- *Values*: general assumptions framing the agile world view
- *Principles*: organizational and technical rules based on those values
- *Roles*: responsibilities and privileges of actors involved
- *Practices*: specific activities based on said principles
- *Artifacts*: virtual and physical tools supporting those practices

1.1 Values

The fundamental assumptions of agile software development are captured in the following values:

1. *Redefined roles for developers, managers, and customers*: Some of the manager's duties are transferred to the team, such as the selection and assignment of tasks. The developers and their code are moved into the center. Customers are no longer passive recipients of a product but active participants in the development process.
2. *No "big upfront" steps*: Activities preceding the writing of code such as gathering requirements ("The customer does not know what he wants!") or creating a design ("The developers do not know what will work!") are left out because they are subject to change anyway. Instead, requirements and design emerge in a continuous process involving the customer, in which the software is iteratively refactored into his acceptance.
3. *Iterative development*: Development takes place in iterations of a fixed time, usually a few weeks, for which the functionality with the highest business value is implemented by working through a prioritized list of tasks. Functionality is added iteratively.
4. *Limited, negotiated functionality*: Only the most important features, measured by their business value, will be implemented. Unused functionality is deemed wasteful and therefore not implemented in the first place. The functionality to be added is negotiated before the start of every iteration. Since it is empirically impossible to fix both scope and deadline, usually the deadline is retained, but the scope limited accordingly.
5. *Focus on quality, understood as achieved through testing*: Quality is ensured by continuous testing rather than by upfront design decisions or by sticking to development methodologies. The project's regression test suite is a central artifact and must always pass when new functionality is added.

1.2 Principles

The following principles—not the ones from the original Manifesto, but extracted from the various texts on agile software development—turn said values into prescriptions:

- Organizational
    1. *Put the customer at the center*: Customer representatives are involved throughout the project, which should deliver the best Return on Investment to the customer.
    2. *Let the team self-organize*: The team members pick their own tasks rather than having them assigned by a manager.
    3. *Work at a sustainable pace*: Programmers work reasonable hours rather than through intense phases with long hours to meet deadlines set by a manager.
    4. *Develop minimal software*: 1) Only essential functions are built (*minimal functionality*); 2) Only what is requested is built, and extra work for future reuse and extensibility is left out (*minimal product*); 3) Only what is delivered to the customer—programs and tests—is built (*minimal artifacts*).
    5. *Accept change*: Requirements are not complete at the beginning, but evolve as customers interact with intermediate releases. Change is considered normal.
- Technical
    1. *Develop iteratively*: Every iteration of a fixed number of weeks produces a working release, providing the customer new functionality he can try out and give feedback on, which is then considered for a later iteration. No new functionality is demanded during an ongoing iteration; such requests are taken into consideration for an upcoming iteration instead.
    2. *Treat tests as a key resource*: Quality is ensured by automated tests. No new development must start until all current tests pass. A text expresses the requirements new code must satisfy and is therefore written before the production code (*test first*).
    3. *Express requirements through scenarios*: A scenario (e.g. a *use case* or a *user story*) describes an interaction of a user with the system that is to be built. Scenarios are obtained from the customer and written from a user's perspective. Unlike requirements, scenarios are not complete specifications but only examples. Scenarios do not have to be collected at the beginning of the project, but can be added and modified during development.

Thus, requirements are replaced by two artifacts: tests and scenarios.

1.3 Roles

Agile methods define various roles:

1. The *Team* is a self-organizing group of developers and other roles. Members of the team assign work items to themselves.

2. The *Product Owner* is responsible for defining the properties of the product under development. This encompasses the right to change those properties, but not while an iteration is ongoing.
3. The *Scrum Master* acts as a coach, mentor, guru, and method enforcer for the team. This role cannot be assumed by the same person that acts as the Product Owner.
4. The *Customer* is directly involved in the project or even a member of the team—rather than just being the source of requirements at the beginning and the recipient of the finished product at the end of the project.

1.4 Practices

The principles stated before are implemented using the following practices:

- Organizational

    1. *Daily Meeting*: Every team member answers the following questions in a daily face-to-face meeting that takes no longer than 15 minutes: 1) "What did I do yesterday?" 2) "What do I plan to do today?" 3) "What impediments am I facing?" Those impediments are then resolved outside the daily meeting in order to keep it short.
    2. *Planning Game*: Work items are estimated in a group setting, where participants are forced to come up with their own initial guess independently. Afterwards, a concensus is found iteratively in a group discussion.
    3. *Continuous Integration*: Changes are integrated continuously (i.e. multiple times per day) rather than after longer development periods in order to detect incompatibilities early on.
    4. *Retrospective*: The team reflects on the finished iteration in order to use the experience gained and lessons learned to improve the upcoming iteration.
    5. *Shared Code Ownership*: The team as a whole, rather than individual programmers, are responsible for the entire code base. This is supposed to reduce the dependence on individuals and to avoid territorial conflicts.

- Technical

    1. *Test-Driven Development*: A yet failing test case is written that describes the new functionality to be added, after which the actual functionality is implemented in order to make the new test case pass. The code can then be refactored as long as all the entire regression test suite passes.
    2. *Refactoring*: The implementation is adjusted structurally in order to improve the design of the system. This step is crucial in conjunction with test-driven development to make sure that the code being written is not only working in terms of passing tests but also well-designed. Refactoring is the agile answer to big upfront design decisions of traditional methods.
    3. *Pair Programming*: Code is developed by two programmers sharing a workstation in changing roles—the "driver" writing the code while explaining his thoughts, and

the "navigator" trying to understand the driver's thought process in order to catch mistakes early on and to provide criticism.
4. *Simplest Solution*: Software engineering principles to improve the code's extensibility and reusability are ignored because—according to agile methods—the further direction of development cannot be known in advance. Instead, only the minimalistic product requested is built and delivered.
5. *Coding Standards*: Teams define style rules that are then applied to all the code being written.

1.5 Artifacts

Agile methods rely on a set of virtual and material artifacts:

- Virtual
    1. *Use Case* and *User Story*: Both are scenarios describing an interaction of a user with a system. Use cases predate agile methods and cover an entire run through the system, whereas user stories originate from agile methods and only describe the interaction from the user's point of view.
    2. *Burndown Chart*: The amount of work items due for an iteration (y-axis) is plotted against time (x-axis) to measure the *velocity*: the amount of work done per unit of time. Since the amount of work items is fixed during an iteration, the plotted line is falling as tasks are finished (and not re-opened). The plotted line can quickly be compared against a planned linear velocity denoted by a straight falling line, which informs team members of their progress (and lack thereof) at a glance.
- Material
    1. *Story Card*: User stories shall be written down to small cards of the same size (3x5 inches in the US, DIN A7 in Europe), which therefore must be kept short.
    2. *Story Board*: The story cards for an iteration are pinned to a story board, where the card's location indicates the underlying user story's progress, for which columns titled "todo", "in progress", "testing", "done" or the like are used.
    3. *OpenRoom*: Development shall take in an open-plan setting as opposed to cubicles or closed offices, which favours interaction between team members instead of isolating them.

1.6 A First Assessment

Some of the ideas stated are genuine inventions of agile methods, while others are older and just have been re-discovered and popularized. Some of those ideas are good, others less so. Combining those qualities—new and old, good and bad—allows for clustering agile ideas into four categories:

- *not new, not good*: User stories and use cases document examples of interactions with a system. While they are useful to validate requirements, they are inadequate for replacing them. Without a proper requirements process, crucial engineering steps such as generalization and abstraction are omitted, resulting in poorly extensible and inflexible products.
- *new, not good*: While pair programming may be a useful technique in some cases, insisting on its constant use (as Extreme Programming does) ignores both different personalities of programmers and studies that fail to show its benefits. Furthermore, methods like Lean denounce useful engineering qualities such as generalization, extensibility, and automation as "waste", which might be appropriate to push back against unproductive perfection in same cases, but causes more harm then benefit when ignored altogether. Leaving out a systematic requirements process because requirements will change anyway is an inappropriate implication, because all kinds of artifacts in a software project are subject to change—but are being created nonetheless.
- *not new, good*: Iterative development with daily integration builds as well as early and frequent releases has been practiced both in commercial settings and in Open-source projects for decades. While agile methods insist on those practices, they have not invented them. The importance of embracing change is also not a new idea, but has been recognized long before. Ironically, some practices advertised by proponents of agile methods make change actually harder rather than facilitating it.
- *new, good*: Allowing a competent team to manage itself is certainly empowering. A daily meeting not only reinforces interaction between programmers, but also clarifies the next steps to be taken, and allows for early detection and resolution of impediments. Time-boxed iterations with frozen requirements during that period add stability to the development process without rendering it inflexible—change is only delayed for a couple of weeks and then handled properly. The focus on the regression test suite as a central artifact of development encourages both quality and professionalism. The focus on code as the project's main deliverable—rather than diagrams and documents—is another idea that agile methods managed to convey to the software industry at large.

2 Deconstructing Agile Texts

Literature advocating agile methods of software development often resorts to intellectually unsound methods to convince its readers.

2.1 The Plight of the Traveling Seminarist

Consider the following example from Mike Cohn's *Succeeding with Agile*, in which the author tries to convince the reader of the superiority of the spoken over the written word:

When the author asked his assistant to "book the Hyatt in Denver" by email, she responded with "the hotel is booked", by which she meant that she was unable to book a room, because the hotel

was already full. However, the author interpreted her response as "your room has been booked as requested".

Cohn then argues that this misunderstanding could have been avoided had this exchange taken place by phone rather than by email. Therefore, he argues, discussions shall be favored over documents.

Ironically, this anecdote was originally presented orally in a seminar. It is way less convincing in the book's written form—where the misunderstanding becomes evident to the reader immediately. Thus, this anecdote refutes its own argument.

It is also an overstretch to conclude that discussions are better suited for software development than written requirements: This is a mere *proof by anecdote*. An anecdote, however, cannot proof a generalization; it can only proof that a generalization does *not* hold by counterexample.

Also, an opposing anecdote of a misunderstanding that could have been avoided by requiring a written specification rather than just relying on the memory of a discussion would already be a sufficient counterargument to the initial "proof" by anecdote.

Usually, such arguments just continue by invoking more anecdotes, or even worse, quotes and aphorisms, but which nothing is ever concluded.

There are many reasons to write down requirements:

- The spoken word is more ambiguous than the written one. The issue of written requirements "looking more precise than they are" is better resolved by more formal notations rather than by resorting to the even more ambiguous spoken language.
- Many discussions end with the request to "write it down", because a final decision often can only be made after reviewing a written request.
- Projects involving people from different cultures with their own English accents often suffer from misunderstandings in verbal discussions that become apparent immediately in the written form, after which the issue can usually be resolved quickly.
- Verbal discussions and their conclusions are only known to the people that attended them. A written version of the conclusions made can be shared with a wider circle.
- If decisions are made on the basis of verbal discussions, it is not certain whether or not the person making the decision is actually authorized to do so. A written document can be reviewed by the relevant stakeholders, unlike decisions being made by the last people that talked about an issue.
- People leave and join ongoing software projects. Unlike verbal discussions, written requirements survive such fluctuations.

Even though pyramids and cathedrals have been built based on mostly verbal interactions, modern engineering came a long way thanks to written requirements. Rejecting them because they are not always 100% precise is the wrong conclusion: more formal written requirements rather than fuzzy discussions are the way out.

Written words *can* be misleading: A written requirement for a system to respond "in real time" says nothing. However, here a more precise language such as mathematics ($t_{\text{answer}} \leq 0.1s$) rather than the more ambiguous verbal communication is the solution.

Communication *is* hard and can cause issues in every projects. Written documents sometimes are just not enough: it's also important for the stakeholders to talk to one another. But this doesn't mean that spoken language can replace the written word, quite the opposite: spoken language *complements* documents.

It's true that written documents are time-consuming and expensive to create. But they can be archived and searched through—unlike verbal communication.

2.2 The Top Seven Rhetorical Traps

Advocates of agile methods often rely on rhetorical devices in their texts. Even though such unsound techniques do not disprove their argument, they should make the reader alert to attempts manipulation. The most common of those rhetorical devices are:

1. *Proof by Anecdote*: While anecdotes can be useful to illustrate a point, they can also backfire when a reader made different experiences than implied with the anecdote. Some anecdotes rely on analogies, which might invoke images in the reader's mind contrary to the author's intention. An anecdote is to a principle what a test is to a specification: it gives one example that cannot necessarily be extrapolated to the general case.
2. *Slander by Association*: Picking the word "agile", whose opposites all have negative meanings and connotations, is rhetorically very effective. Likewise, lumping all non-agile ideas together with things generally perceived as outdated and backwards ("Waterfall!") might be just as effective—but also dishonest.
3. *Intimidation*: Some ideas initially considered absurd turned out to be genious. However, this does not imply that *any* idea sounding absurd is equally valid. This logical fallacy follows the form "B implies A from A implies B" or "*anything* with quality X is good, because *something* with quality X is good". Any expert's criticism is rejected based on anecdotes in which experts have been proven wrong. Using the establishment as the enemy can be an effective way to rally others behind a revolutionary idea. And if evidence contrary to such an idea shows up, simply the environment, in which the idea has been tried, can be blamed: If agile does not work in your organization, your organization is the problem. Anybody not buying into the agile way of handling projects is labeled as an authoritarian member of the command and control gang.
4. *Catastrophism*: Advocates of a new method need to point out the flaws of existing ones. The term "software crisis" became prevalent in 1968 and has been used ever since to paint a bleak picture of software development and its results. However, a lot of software projects are successful, and the software created thereby performs its duty observably well. Not everything is as bad with the current state of software and the—"pre-agile"—process that created it as proponents of agile methods often claim.

5. *All or Nothing*: It is fair for a methodology to define a set of principles its users have to adhere to in order to make it work. However, such a set of absolute requirements must be kept small for a methodology to be practicable. Blaming failed projects for not totally sticking to the prescriptions while celebrating successful projects cherry picking just a couple of such practices of a methodology is dishonest. One recipe does not fit every organization and project; principles and practices have to be selected as it fits.
6. *Cover Your Behind*: Many agile texts advocate extreme ideas and back those up with some words of warning where those ideas will not necessarily work—but fail to provide useful criteria for such a delimitation. Such tempering serves the authors well for covering their backs ("I didn't tell you to *not* supervise the team/gather requirements/plan the project!"), but leaves the reader without any guidance whether or not he shall implement a particular idea in his specific circumstances. This rhetorical device is often stated as "X does not mean Y" when X and Y are hardly distinguishable. One example from *Lean* claims that "deciding as late as possible does not mean procrastinating"—where "deciding as late as possible" is the literal dictionary definition of "procrastinating".
7. *Unverifiable Claims*: Many agile claims—"doing twice the work in half the time", i.e. improving productivity by a factor of four—are simply too good to be true. While it's possible that a formerly demotivated team run by incompetent managers can become hugely productive over night when given the chance to work on their own accord, such examples cannot be generalized to all circumstances. Software projects are not repeatable lab experiments: they are influenced by their surrounding environments and people working at them as well as by their experiences made in the process.

3 The Enemy: Big Upfront Anything

Agile methods reject anything "plan-based" or "predictive". Such approaches are lumped together with the term "waterfall" to denounce it using slander by association, thereby ignoring that the original 1970 paper used the Waterfall Model as a strawman argument how software development does *not* work. This view also ignores that engineering is by definition predictive to some extent, using methods of science and management to some degree.

3.1 Requirements Engineering

Software engineering is not just programming. It also involves *requirements analysis*: figuring out the problem a stakeholder wants to solve and appropriate solutions to it. While most of software engineering is about building the system right, requirements are about building the right system.

A big part of requirements engineering is *requirements elicitation*: gathering user needs. This encompasses, among others, stakeholder interviews (asking people what they need) and workshops (discussing and clarifying requirements to identify and resolve conflicts).

The results of this process are usually gathered in a *requirements document*. Proponents of agile methods criticize both process and result using two different arguments:

1. *Waste*: A requirements document is not a useful deliverable for the customer. Some gathered requirements will not be implemented, and gathering them is therefore wasteful.
2. *Change*: Customers do not know what they want and will change their minds as they get a chance of interacting with the system. Requirements become obsolete; it is better to create new versions of the software iteratively based on user feedback than to lock down their initial ideas.

There are two approaches to reduce waste: The plan-first approach reduces waste by first gathering and prioritizing requirements and then dropping the ones considered not essential. The agile approach reduces waste by producing a minimal solution, of which parts are dropped the customer deems unnecessary.

While the first approach produces one sort of waste—unused requirements—it avoids another form of waste—unused implementations, which are common in agile projects—and can be very frustrating for developers. Both techniques have the same goal and can be used in conjunction.

Large and detailed requirements documents are overkill for many projects. This does not imply that no requirements shall be gathered whatsoever, but that the level of detail needs to be adjusted to fit the project. And even though a requirements document is not a deliverable, it serves as the foundation of a *system documentation*, which is delivered to the customer.

The argument concerning change is correct in so far as requirements cannot be frozen after initially gathering them. However, having gathered an initial set of requirements does not imply carving them in stone. Requirements are just one of many artifacts of a software project alongside code and regression tests—which change all the time and are written nonetheless! Just because plans need to be adjusted does not mean that plans are useless.

Furthermore, an initial requirements *phase* does not imply that the resulting requirements *document* must remain static. Requirements can be put under version control along the code that implements them.

A proper requirements process distinguishes between two kinds of requirements describing different aspects:

1. *Domain*: properties of a model as part of the world in which the system will operate
2. *Machine*: desired properties of the system the project wants to build

As agile methods see requirements as design or candidate solutions, they fail to make this important distinction: It is *not* up to the project to define the domain, which is given as business rules, regulations, laws, or properties of the physical world. Such restrictions are *gathered* rather than made up. The project is free, however, to define the machine requirements—as design decisions, so to speak.

The agile critique is right that often too much time is spent on figuring out design and implementation details to be disguised as mere requirements. It's often better to wait until more reliable information becomes available.

3.2 Architecture and Design

In software, the code expresses the actual solution. The *design* of a software defines its *architecture*: its modular structure, the abstractions chosen, the design patterns used, and the interfaces specified.

It has long been recognized that in software development, there is no sharp distinction between design and implementation, because the implementation has major impacts on the properties of the system as a whole, e.g. on its performance.

Compared to an industrial process, the compilation rather than the writing of the source code comes closest to the production phase. Therefore, the word "design" has a different meaning in software development than in, say, mechanical or electrical engineering.

There is also no firm line between code and documentation, given that code contains comments, and artifacts such as UML diagrams resemble code quite closely to be useful.

Code always has an architecture—as a result of a distinct design phase or as it emerged from development. Just like a mathematical paper that looks much orderlier than the process that created it, a software design can be the result of a more or less structured approach.

While agile methods denounce a separate upfront design phase, they have no common approach to design, but some guiding principles:

1. Design in iterations alternating with regular implementation iterations rather than in a separate upfront design phase.
2. Solve the problem at hand rather than focussing on extensible and reusable design.
3. Create a working solution and refactor it rather than aiming at a perfect initial solution.

The agile literature admits that approach makes planning the work and assigning it to team members much harder. Not having a big architecture picture can also make the work on the system under development harder. Reworking its parts becomes inevitable, too.

This view on design is sensible insofar as it is not a good idea to do too much design at the beginning of a project. However, the categorical rejection of any upfront design activity also has its issues:

1. Refactoring only works well in a code base with a decent design to begin with. This is hardly the case if no thoughts are spent on extensibility and reusability at all.
2. Some design decisions—or the failure to take them—are extremely costly to revert, e.g. the decision between a monolingual and a multilingual user interface.

Just like security, good design requires both some upfront conceptual work and an ongoing effort. Here, the total agile rejection of any upfront design process goes too far.

3.3 Lifecycle Models

Lifecycle models specify and standardize the sequence of phases through which a software projects proceeds: analysis, implementation, verification and validation, etc. Such models can proceed sequentially (like a waterfall) or iteratively (like a spiral) and are usually depicted using boxes (for phases) and arrows (for transitions). They are both *descriptive* (showing how successful teams work) and *prescriptive* (saying how teams should work).

Even though such models like the Waterfall or the V-Model are (often: rightfully) rejected, they can be useful for different reasons. Historically, such models were a healthy reaction to a totally unstructured and chaotic "hacking" process, which had to be separated into different phases and activities. Conceptually, separating development into distinct activities is still useful after rejecting their separation into different temporal phases. Pedagogically, such idealized models are a useful device to demonstrate the need for more flexible approaches.

While agile software development rejects the Waterfall model for good reasons, it fails to see how agile methods such as Scrum also prescribe a lifecycle model with distinct development, planning, and review activities or even phases. Every project needs a temporal framework to predict and assess its progress, be it a sequential or iterative model.

One such approach is the *Rational Unified Process* (RUP) that combines practices (iterative development, managing requirements, component-based development, visual modelling of software, continuous quality verification, and controlling changes) and phases (inception, elaboration, construction, and transition—i.e. deployment). Even though these phases are a bad fit for an agile project, the practices can be adopted to an iterative approach.

3.4 Maturity Models

Maturity models such as the ones described in the ISO 9000 set of standards or the more software-specific *Capability Maturity Model (Integration)*, short: CMM(I), have been introduced to the software industry since the 1980s. Even though they might appear as monstrosities due to the huge documents in which they are described, they introduce some useful concepts.

CMMI is a collection of best practices that allows an organization to reach identified goals and assess its compliance. While some of those goals and practices are generic ("institutionalized and planned processes"), most of them focus on a specific area such as configuration management, project planning, or risk management.

The focus of CMMI is on the process rather than on the product: it does not ensure that there are no bugs in the software, but makes sure a process is in place to deal with those bugs.

CMMI defines five levels of increasing maturity:

1. *initial*: The process is ad hoc and chaotic; success depends on heroic deeds of individuals.
2. *managed*: Processes exist and are supported by their stakeholders.
3. *defined*: Processes are precisely defined for the entire organization.

4. *quantitatively managed*: Processes are assessed using numeric metrics.
5. *optimizing*: Processes include mechanisms for their continuous improvement via feedback loops.

CMMI allows organizations such as the US Department of Defense (DoD) to choose its suppliers using objective criteria. It also allows outsourcing companies to establish credibility in a foreign market.

The *Personal Software Process* (PSP) and *Team Software Process* (TSP) are approaches to systematically apply good practices on the individual and team level.

Even though such models do not contradict agile ideas—both CMMI and agile codify processes and practices—maturity models are often rejected as too rigid or even as wasteful. While this rejection is understandable for the planning part of CMMI, its practices can be adopted to agile approaches.

Proponents of agile methods criticize CMMI for creating bias against change due to its emphasis on processes. It also transfers authority from development to managers, which is, however, rather an issue of bureaucratic organizations rather than of the model itself.

CMMI is often introduced for regulatory reasons or due to commercial incentives. It is possible to combine its ideas with agile approaches; adaptation of CMMI does not imply rejecting agile ideas. Agile adaptations of such maturity models are testamtent to this. The *Shu-Ha-Ri* scale is a three-step gradation adopted from Japanese martial arts:

1. *Shu*: to *obey*, recipies are learned and applied.
2. *Ha*: to *detach*, actions are abstracted from core rules and combined.
3. *Ri*: to *surpass*, actions go beyond existing rules and devise own solutions.

This gradation is arguably compatible to the western bachelor-master-PhD scale—albeit without the exotic far-eastern appeal.

4 Agile Principles

In software development, a *principle* is a methodological rule from which specific *practices* can be derived. A good principle is:

1. *abstract* as a foundation for specific practices—rather than stating a practice by itself,
2. *falsifiable* to be interesting—rather than a platitude everybody agrees on, and
3. *prescriptive* to guide software development—rather than only describing it.

The agile manifesto provides twelve principles, which are hereby stated and rated according to the three criteria mentioned above:

1. Our highest priority is to satisfy the customer through early and continuous delivery.

- While "satisfy the customer" and "valuable software" sound like platitudes, "early and continuous delivery" is abstract, falsifiable, and prescriptive: hence, a valid principle.

2. Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
   - This attitude to welcome rather than oppose change is a valid principle.
3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference for the shorter timescale.
   - Altough partially redundant with the first principle, this is still valid.
4. Business people and developers must work together daily throughout the project.
   - Abstract, falsifiable, prescriptive—a perfectly valid principle.
5. Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job right.
   - This is clearly a platitude. Nobody in their right mind would build projects around unmotivated individuals, deny them the resources they need, or give them a job while not trusting them to get it right.
6. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
   - This is rather specific and therefore not a principle but a practice.
7. Working software is the primary measure of progress.
   - This should be rephrased in a more prescriptive manner, but otherwise is a valid principle, despite its slight redundancy with the third one.
8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
   - While striving for sustainable development rather sounds like a platitude, maintaining a constant pace over a longer period of time is a valid principle in an industry that resorts to "crunch mode" to finish late projects.
9. Continuous attention to technical excellence and good design enhances agility.
   - This is the most blatant platitude: Nobody in their right mind can argue against "technical excellence" or "good design".
10. Simplicity—the art of maximizing the work not done—is essential.
    - Those are rather *two* principles, because simplicity and minimalism are strongly related, but not the same thing.
11. The best architectures, requirements, and designs emerge from self-organizing teams.

    - Prescriptive, abstract, and falsifiable—another valid principle.

12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behaviour accordingly.

    - This is rather a practice in accordance with the eleventh principle.

Only 2, 4, 8, and 11 are both valid and *independent* principles; 10 states two principles. 1 and 3 as well as 3 and 7 (which needs rephrasing) are valid but somewhat *redundant* principles. 5 and 9 are platitudes; 6 and 12 are practices. Not a single statement mentions "testing"—a core property of all agile methods, around which many practices revolve.

Therefore, the list of principles already introduced in the first chapter shall be used for further discussion instead:

- Organizational

    1. Put the customer at the center.
    2. Let the team self-organize.
    3. Work at a sustainable pace.
    4. Develop minimal software.
    5. Accept change.

- Technical

    1. Develop iteratively.
    2. Treat tests as a key resource.
    3. Express requirements through scenarios.

4.1 Put the Customer at the Center

Traditional approaches limit customer interaction to the initial requirements phase and the final acceptance tests with no customer involvement in between. In an agile project, the customer is invited to regular project meetings, can interact freely with developers, is allowed try out intermediary versions of the software being built, or is even embedded into the development team.

This practice is supposed to resolve one of the biggest issues in software projects: delivering working software that doesn't fit the customer's needs. As a NASA study revealed, the main reason for this issue is the development team failing to understand the requirements.

While constantly interacting with the customer is deemed as the appropriate counter measure, putting more effort into a proper requirements process is rejected by proponents of agile software development, even though it arguably is the more effective solution to this problem.

Constant interactions with a single representative from the customer cannot replace a proper requirements process, which has to deal with many different stakeholders with different needs, views, priorities—and contradicting requirements.

Embedding a single customer representative into the development team for the entire project can skew the team's understanding towards the biased perspective of that particular individual. The customer is usually not willing to take away the most qualified employee from his regular tasks for an extended period of time. Gathering the most competent individuals for a couple of requirements workshops is rarely an issue for the customer, though.

4.2 Let the Team Self-Organize

Agile approaches shift duties such as the assignment of tasks from managers to the team. Scrum even dissolves the role of a project manager and moves his duties over to a product owner (responsible for the product) and a Scrum master (coaching the team and enforcing the method).

The direct command and control approach of the classic manager is replaced by peer pressure and other means of subtle control. The agile literature is unclear about the remaining duties of the manager and mostly defines what a manager is *not* supposed to do. Besides deciding what projects are being done and who is working on them, a manager is supposed to guide the team towards self-organization, thereby rendering his own position obsolete.

While a competent team can suffer and underperform when guided by an incompetent manager, a group of inexperienced developers will not be able to manage itself without any supervision. It requires an experienced team of highly competent programmers to do that.

A pragmatic approach consistent with the agile literature is a manager encouraging initiative from the team members in order to gradually evolve its management style from command and control to self-organization.

4.3 Work at a Sustainable Pace

Agile is the antidote to *death marches*—projects with unrealistic deadlines and an ever-growing list of fuzzy requirements that can only be finished using pressure from management and a lot of extra hours. Instead, agile approaches place the programmers at the center and allow them to work under conditions where they can unfold their full potential.

The human factor in software development has been discussed in books pre-dating the agile literature such as *PeopleWare* published in 1987. The agile's preference of face-to-face communication over written language and code over management-centric artifacts like plans, models, and documents goes back to this rather humanistic perspective.

The Marxist undertone of earlier literature was replaced by a focus on ROI and other capitalistic goals in agile texts. while XP and Crystal remain true programmer-pride movements, Scrum and Lean are rooted in industrial production and aim to maximize productivity by minimizing waste.

Some texts promote rather ruthless management techniques, others emphasize concepts like *slack*—including a few uncritical minor task in every iteration, which can be easily dropped when the schedule tightens.

4.4 Develop Minimal Software

Agile methods strive for simplicity by getting user feedback quickly based on small increments delivered frequently to the customer. This simplicity manifests itself in several forms:

1. *Produce minimal functionality*: Superfluous features not only waste precious development time, they also create a maintenance burden. Once delivered, a feature cannot be removed if even a single user insists on keeping it. The slogan YAGNI—"You ain't gonna need it!"—is the antidote to bloat and helps to only include features that are needed *now*.
2. *Produce only the product requested*: While software engineering traditionally values *extendibility* (building an architecture that supports future extensions) and *reusability* (building components as general as possible for later use elsewhere), agile methods focus on the here and now. It is true that reusability requires additional work such as packaging, documentation, and training; and therefore might be distractive from the project's immediate goals. However, leaving out exception handling, input validation, and useful error messages in order to do "the simplest thing that could possibly work", as some authors suggest, leads to code that can only handle the specific cases described in the respective scenarios and falls apart when presented with other inputs.
3. *Produce only code and tests*: Agile methods see artifacts such as requirements, designs, plans, and documentation as diversions from doing the actual work—writing code and tests. Such artifacts that are not part of the end product must prove their purpose lest they are dropped. Some agile authors—grudgingly—admit that usually a lot more than just the code is delivered to the customer, i.e. documentation and training material. (One might wonder if and how tests are delivered to the customer.)

Many products suffer from too many features, and a lot of paperwork becomes obsolete the moment it is written. This is, however, the result of *bad* management. A classic upfront requirements analysis establishes the priorities of different stakeholders before any work is produced. If every stakeholder is given some amount of virtual money to express his priorities with, critical features quickly emerge while optional ones are dropped immediately.

The maxim of "building the simplest thing that can possibly work" leads to picking the low hanging fruit. This produces presentable results with convincing demos, but the hard problems tend to be postponed until there's no more time and budget left. A system suffering from severe performance issues cannot be made fast by some iterations of refactoring, an entire redesign is often required instead. An upfront thinking process prevents such costly endeavours.

Proper risk management helps establishing critical tasks right at the beginning of the project—before convincing presentable results of uncritical features are shown to the stakeholders, while the project is headed for disaster because the critical decisions are postponed. Such a project is best

stopped early after considering the risks involved. Responsible engineering prioritizes the essential over the visible.

4.4.1 Complexity

There are two different kinds of complexity:

1. *Additive* Complexity: The basic problem is simple, and details can be added one by one. Think of a common case and several special cases, which are mutually exclusive, such as different VAT rates for different kinds of products.
2. *Multiplicative* Complexity: The basic problem is already hard to solve, and there is no acceptable solution until the problem has been solved in its entirety. Think of a mechanism that needs to be fully established before it becomes useful, such as multilingual user interfaces. (Establishing the mechanism is of multiplicative, adding another language of additive complexity.)

If different features of a system are mutually independent, then there's only additive complexity. If, however, those features are entangled and interact with one another, there's multiplicative complexity.

Consider the "busy treatment" mechanism of a telephony system. Every rule has a pre-condition, a priority, and an action. Such requirements cannot be implemented one by one as unrelated scenarios, but require an entire system to deal with conflicting rules. (Think of mutually forwarded calls, or overwriting "do not disturb" mode for emergency calls.)

Features can only be implemented one by one, if their complexity is additive. Features with multiplicative complexity require an upfront and systematic process for analysis and design.

4.4.2 Documents

The argument that customers or users are not interested in documents might be true—but also weak. Certainly the end user also cares little about the code or test cases, but those are produced nonetheless. A house owner might care little about the blueprints and building permits, but building the house certainly required those artifacts, and many stakeholders cared a lot about them. The question is less if the end user cares, but if the developers required to maintain and extend the system do.

It is true that things can change, and documents and reality are hard to keep in sync. Unlike buildings or cars, software can be changed all the time at relatively modest costs. Dismissing an entire category of artifacts on the basis that those are subject to change is a weak argument, because code is also changed all the time and written nonetheless.

4.4.3 Simplicity

Simplicity is often hard to achieve and requires additional work—counter to the agile mantra that simplicity is "maximizing the amount of work not done". Proponents of simplicity, such das Dijkstra, Wirth, and Hoare, are also advocates of rigorous methods involving mathematical models and upfront thinking—certainly do not care about "maximizing the amount of work not done" in order to produce quick presentable results.

In his *Plea for Lean Software* from 1995, Wirth sees the price to achieve simplicity "in a clear conceptual basis and a well-conceived appropriate system structure"—clearly big upfront tasks. This is the antidote to the agile method called "Lean", which advocates for postponing (design) decisions while building the system feature by (presentable) feature.

4.5 Accept Change

Requirements change over time, especially if the customer is directly involved in the project, as agile methods insist on doing. The Agile Manifesto not only accepts but even welcomes change. This is remarkable, because changes tend to require additional work, especially if they concern features that already have been implemented.

In practice, agile methods limit change. Scrum's *closed window* rule insists that change only happens outside of sprints. During a sprint, the team won't accept any change requests.

As opposed to the caricature the agile crowd draws of traditional software development, the importance of dealing with change has long been recognized. Traditional methods just emphasize that change has to be managed properly. This requires software being designed for extendibility. So the problem of change is rather technical than psychological.

The agile idea of having a regression test suite, which is especially promoted by Extreme Programming, helps dealing with change by minimizing the risk of undetected regression errors.

Unfortunately, the agile insistence on programming just for the "here and now" that deems creating an extensible architecture wasteful rather impedes change than enabling it. A software design that supports change requires upfront thinking, which the agile literature rejects.

4.6 Develop Iteratively

Agile projects are handled iteratively without any upfront requirements or design phase. There are two approaches to iterative development:

- The *vertical* approach produces different layers iteratively: from persistence over networking and business logic to user interface. This technology-focused approach only delivers an end-to-end user experience late in the project.

- The *horizonal* approach produces a working system in every iteration, providing a new end-to-end user experience frequently at the expense of the completeness of the technical mechanisms involved.

Agile methods favour the horizontal approach. This dichotomy is related to the multiplicative and additive kinds of complexity, with the former being better suited for the vertical and the latter for the horizontal—agile—approach.

Agile projects are split up into iterations of the same duration, typically a couple of weeks up to a (calendar) month. The deadline is firm, but the scope is more flexible: What is not finished during an iteration is moved to a later iteration or dropped entirely.

When developers make their own estimates and are not allowed any extra time, progress becomes more predictable, and the developers strive to deliver what they have promised.

Constraints of external customers still apply, so it is hardly ever permissible to postpone unfinished tasks for multiple iterations.

"Big Bang" approaches have a bad track record: A team diverging from the customer for a long time will make inconsistent assumptions and ultimately come back with a product not suiting the customer's needs.

Frequent iterations such as a nightly build that must not break, are a reasonable approach to prevent this kind of divergence. However, the insistence on frequent *working* iterations can be wasteful, because it requires keeping up a facade of visible progress while the architecture might crumble in the background.

Just like other engineering efforts, software requires a solid foundation. Therefore, a compromise between engineering work done in the background and an enhanced user experience delivered in the foreground must be found. Neither a fragile system with many features nor a perfect engineering effort without any user interaction will satisfy the customer.

4.6.1 The Order of Tasks

In which order shall tasks be tackled? Risk management suggests to do the hardest parts first, so a project bound to fail does so quickly without wasting a lot of resources.

However, the agile literature argues against this, because early failure might depress a promising team, which might otherwise become able to pull off hard things once it started working together properly and managed to build up the required motivation from succeeding with some easier tasks.

While this is sound advice for a team working the first time together in a given configuration, experienced teams should not build up unjusified confidence by delivering some trivial functionality and postponing the hard work until there is no time left for the project.

The agile literature suggests to pick up the tasks with the highest business value first. This might sound attractive to some types of managers but ignores that features are implemented as systems,

and the architecture of such a system must also support the features with the second and third highest business value.

A possible solution to this conflict is *dual development*, in which architecture work in the background and delivery of the end-to-end user experience in the foreground is done seperately. This can happen in distinct phases or during the same time by different members of the team.